

CivicDesk

Module 2.1 — Identity & Access Management (IAM)

Complete Technical Reference Documentation

Spring Boot 3.4 · Spring Security 6 · JWT (HS256) · MySQL 8 / H2

Prepared for developers, architects, and stakeholders
Document version 1.0

Table of Contents

[The table of contents is generated automatically by Microsoft Word. Right-click it and choose "Update Field" if page numbers need refreshing.]

1. Executive Summary

Table of Contents.....	1
1. Executive Summary.....	1
1.1 What This Project Is.....	4
1.2 Core Capabilities.....	4
1.3 How to Read This Document.....	5
2. Technology Stack & External Dependencies	5
2.1 Stack Overview	5
2.2 Maven Dependencies (pom.xml)	6
3. Project Structure & Layered Architecture	7
3.1 Folder Structure	7
3.2 The Layered Architecture	8

3.3 Separation of Concerns — Why This Structure?	9
4. Configuration & Application Bootstrap	9
4.1 CivicDeskApplication — the Entry Point	9
4.2 SecurityConfig — the Heart of Authentication/Authorization	10
4.3 JwtConfig — Typed JWT Settings	11
4.4 CorsConfig — Cross-Origin Policy	11
4.5 OpenApiConfig — Swagger / API Documentation	11
4.6 DataSeeder — Startup Bootstrap Data	12
5. The Common Layer (Shared Building Blocks)	12
5.1 Response Envelopes (<code>common/response</code>)	13
5.1.1 ApiResponse	13
5.1.2 ErrorResponse	13
5.1.3 PageResponse<T>	13
5.2 Exception Handling (<code>common/exception</code>)	14
5.2.1 GlobalExceptionHandler	14
5.2.2 The Domain Exception Family	15
5.3 Utilities (<code>common/util</code>)	15
5.3.1 JwtUtil — Token Authority	15
5.3.2 NationalIdUtil — PII Protection	15
5.3.3 SecurityContextUtil — Reading the Current User	15
5.3.4 ClientIpUtil — Resolving the Client IP	16
5.4 Custom ID Generation (<code>common/id</code>)	16
NumericStringSequenceGenerator	16
6. The Data Layer: Entities, Enums, Repositories & Schema	16
6.1 Cross-Cutting Persistence Design	16
6.2 Entity: User	17
6.3 Entity: AuditLog	18
6.4 Entity: Department	19
6.5 Enums	19
6.6 Repositories (<code>module/iam/repository</code>)	19
6.7 Specifications (<code>module/iam/repository/spec</code>)	20
6.8 Database Schema (<code>src/main/resources/db/civicdesk_iam.sql</code>)	20
7. Data Transfer Objects (DTOs)	21
7.1 Why DTOs Instead of Exposing Entities	21
7.2 Request DTOs (<code>dto/request</code>)	21
7.3 Response DTOs (<code>dto/response</code>)	22

8. The Service Layer (Business Logic).....	23
8.1 Why a Service Layer Exists.....	23
8.2 AuthServiceImpl.....	23
8.3 UserServiceImpl.....	24
8.4 AuditServiceImpl.....	25
8.5 DepartmentServiceImpl.....	25
8.6 Service-Layer Summary.....	25
9. The Controller Layer (REST API).....	26
9.1 Role of Controllers & the “Thin Controller” Principle.....	26
9.2 Common Controller Annotations.....	26
9.3 AuthController.....	27
9.4 UserController.....	27
9.5 AuditLogController.....	28
9.6 DepartmentController.....	28
10. Security & JWT Authentication (In Depth).....	28
10.1 What Is a JWT?.....	29
10.2 The Three Parts of a JWT.....	29
10.3 How HS256 Signing Works.....	29
10.4 Token Generation & Verification — JwtUtil.....	29
10.5 The Per-Request Gatekeeper — JwtAuthFilter.....	30
10.6 Why JWT Is Stateless (and Why It Matters Here).....	31
10.7 The Authorization Process (RBAC).....	31
10.8 Advantages & Limitations of This Design.....	31
11. End-to-End Flows.....	32
11.1 Generic Request Flow (API call → Controller → Service → DB → Response).....	32
11.2 Authentication Flow (Login → Token).....	32
11.3 Staff Onboarding Flow (Set-Password on First Login).....	33
11.4 Data Flow Between Layers.....	33
12. Complete Annotation Reference.....	33
12.1 Core Spring / Stereotypes.....	33
12.2 Web / MVC.....	34
12.3 Security.....	35
12.4 JPA / Hibernate (Persistence).....	35
12.5 Validation (Jakarta Bean Validation).....	36
12.6 Jackson (JSON).....	36
12.7 Lombok — Note.....	36
12.8 Testing Annotations.....	37

13. Error Handling, Logging & Configuration	37
13.1 Error-Handling Strategy.....	37
13.2 Logging Practices.....	37
13.3 Configuration Files	37
13.4 External Dependencies (Runtime Services).....	38
14. Testing (JUnit & Mockito).....	38
14.1 Purpose of Testing & the Test Pyramid.....	38
14.2 Testing Annotations Explained.....	38
14.3 Mockito & Why Mocking Is Required	40
14.4 The Test Suite, File by File.....	40
14.5 Unit vs Integration — A Deliberate Division of Labour	42
15. Appendix: Endpoint Reference.....	42
Roles Reference	43

1.1 What This Project Is

CivicDesk is a civic-services platform (the kind a municipal corporation uses to handle citizen grievances, service requests, permits, and inspections). This document describes **Module 2.1 — Identity & Access Management (IAM)**, the foundation module on which every other module depends.

The IAM module answers three questions for the entire platform:

- **Who are you?** — authentication of citizens and staff (login, registration, password setup).
- **What are you allowed to do?** — authorization through role-based access control (RBAC).
- **What did you do?** — an append-only audit trail of security-relevant actions.

1.2 Core Capabilities

- **Citizen self-registration** — a member of the public creates their own account.
- **Two separate login portals** — one for citizens, one for staff, each rejecting the other's users.
- **JWT issuance** — on successful login the server returns a stateless, signed token (HS256, 30-minute validity) that the client presents on every later request.
- **Staff onboarding** — administrators create staff accounts without passwords; the staff member sets their own password on first use.
- **Role-based administration** — an Admin can create Department Supervisors; a Supervisor can create field staff within their own department.
- **User lifecycle management** — activate, suspend, or deactivate accounts.

- **Audit logging** — every register / login / logout / user-create / status-change / password-set event is recorded with the actor, action, module, IP address, and timestamp.
- **Department directory** — six seeded municipal departments to which staff are assigned.

1.3 How to Read This Document

The document moves from the “big picture” to the smallest detail:

- **Sections 1–3** — what the system does, the technology stack, and the layered architecture (the “why” behind the folder structure).
- **Sections 4–9** — a layer-by-layer walkthrough of *every* source file: configuration, the shared “common” layer, the data layer (entities/repositories), DTOs, services, and controllers.
- **Section 10** — an in-depth treatment of security and JWT authentication (the heart of an IAM module).
- **Sections 11–14** — end-to-end flows, a complete annotation reference, cross-cutting concerns (errors, logging, configuration), and the testing strategy.

Each file is documented under four lenses: **WHAT** it does, **WHY** it exists, **HOW** it works internally, and **HOW it connects** to the rest of the system.

2. Technology Stack & External Dependencies

2.1 Stack Overview

Concern	Technology	Why it was chosen
Language / Runtime	Java 21	Modern LTS Java; records, pattern matching, performance.
Application framework	Spring Boot 3.4.1	Auto-configuration, embedded server, opinionated defaults — minimal boilerplate.
Security	Spring Security 6	Industry-standard authentication/authorization framework; lambda DSL and method security.
Persistence	Spring Data JPA (Hibernate)	Object-Relational Mapping; repositories remove hand-written SQL/JDBC boilerplate.

Database (runtime)	MySQL 8	Reliable, widely-deployed relational database for production.
Database (tests)	H2 (in-memory)	Fast, disposable database so tests need no external server.
Tokens	JWT 0.12.3	Mature library for creating and verifying JSON Web Tokens.
Password hashing	BCrypt (cost 10)	Adaptive, salted one-way hashing — the standard for password storage.
API documentation	springdoc-openapi 2.7.0 (Swagger UI)	Interactive, always-up-to-date API explorer.
Validation	Jakarta Bean Validation	Declarative request validation via annotations.
Testing	JUnit 5, Mockito, Spring Security Test	Unit, slice, and full-stack integration testing.
Build tool	Maven	Dependency management and build lifecycle.

2.2 Maven Dependencies (`pom.xml`)

The project inherits from `spring-boot-starter-parent` 3.4.1, which centralizes dependency versions. The declared dependencies are:

Dependency	Scope	Purpose
<code>spring-boot-starter-web</code>	compile	REST controllers, embedded Tomcat, Jackson JSON.
<code>spring-boot-starter-security</code>	compile	Authentication, authorization, the filter chain, BCrypt.
<code>spring-boot-starter-data-jpa</code>	compile	JPA/Hibernate, repositories, transactions.
<code>spring-boot-starter-validation</code>	compile	Bean Validation (Hibernate Validator) for request DTOs.

<code>mysql-connector-j</code>	runtime	MySQL JDBC driver (only needed at run time).
<code>jjwt-api</code> / <code>jjwt-impl</code> / <code>jjwt-jackson</code> (0.12.3)	compile / runtime	JSON Web Token creation and verification (API + implementation + JSON binding).
<code>springdoc-openapi-starter-webmvc-ui</code> (2.7.0)	compile	Generates OpenAPI spec and serves Swagger UI.
<code>spring-boot-starter-test</code>	test	JUnit 5, Mockito, AssertJ, Spring Test, MockMvc.
<code>spring-security-test</code>	test	Security testing helpers (<code>@WithMockUser</code> , etc.).
<code>h2database</code>	test	In-memory database for tests.

Version note: the project was scaffolded with a newer Spring Boot, but the `pom.xml` is deliberately pinned to **3.4.1** so that the documented APIs (the lambda security DSL, `@EnableMethodSecurity`, and JWT 0.12.3) compile exactly as written. The `spring-boot-maven-plugin` packages the app into an executable JAR.

3. Project Structure & Layered Architecture

3.1 Folder Structure

```

com.civicdesk
├── CivicDeskApplication.java      ← application entry point (main)
├── config/                      ← cross-cutting configuration (beans)
│   ├── SecurityConfig.java      the security filter chain + password encoder
│   ├── JwtConfig.java           typed holder for app.jwt.* properties
│   ├── CorsConfig.java         Cross-Origin Resource Sharing policy
│   ├── OpenApiConfig.java       Swagger / OpenAPI metadata + bearer scheme
│   └── DataSeeder.java          seeds departments + first admin on startup
├── common/                     ← shared, module-agnostic building blocks
│   ├── exception/              GlobalExceptionHandler + domain exceptions
│   ├── response/               ApiResponse · ErrorResponse · PageResponse
│   └── util/                   JwtUtil · NationalIdUtil ·
│                               NumericStringSequenceGenerator (custom ID
│                               generator)
├── module/
│   └── iam/                     ← the Identity & Access Management module
│       ├── controller/         AuthController · UserController ·
│       └── AuditLogController · DepartmentController

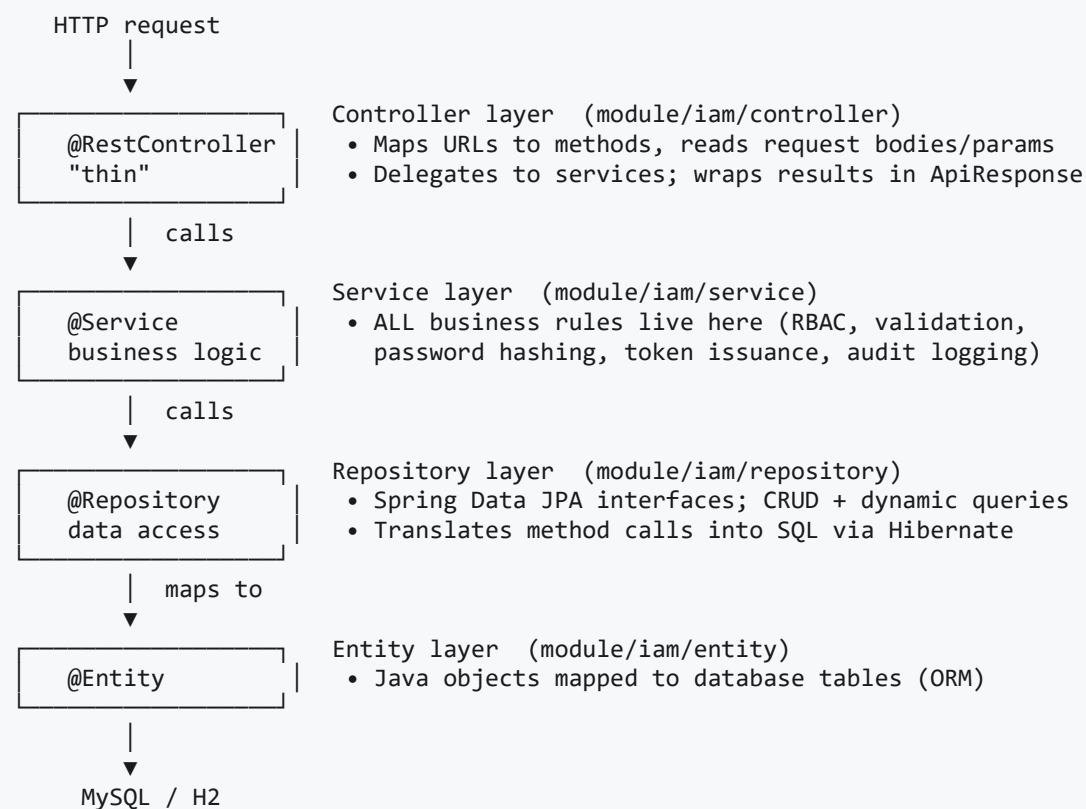
```

<pre> — service/ + Impl each) — repository/ DepartmentRepository — spec/ (dynamic queries) — entity/ — dto/ — request/ CreateUser · SetPassword · UpdateUserStatus — response/ DepartmentResponse · AuditLogResponse — enums/ — security/ </pre>	<pre> Auth / User / Audit / Department (interface UserRepository · AuditLogRepository · UserSpecifications · AuditLogSpecifications User · AuditLog · Department (JPA entities) Register · CitizenLogin · StaffLogin · AuthResponse · UserResponse · Role · UserStatus · AuditAction · AuditModule JwtAuthFilter (per-request JWT validation) </pre>
---	--

The `module/` folder is deliberately a container for many modules. Teammates' modules (`citizen/`, `servicerequest/`, `permit/`, `grievance/`, `analytics/`) live as siblings of `iam/`. The `common/` and `config/` packages are shared by all of them.

3.2 The Layered Architecture

The code follows a classic **layered (n-tier) architecture**. A request flows downward through the layers and a response flows back up:



DTOs (dto/request, dto/response) cross the Controller↔Service boundary so database entities are never exposed directly to the outside world.

3.3 Separation of Concerns — Why This Structure?

Layer	Single responsibility	Benefit
Controller	HTTP concerns only (routing, status codes, serialization).	Easy to read; swap transport without touching logic.
Service	Business rules and orchestration.	Reusable, independently testable, transactional.
Repository	Database access.	No hand-written SQL; database can evolve behind the interface.
Entity	The persistent data model.	One source of truth for the schema mapping.
DTO	The public API contract.	Security and decoupling — internals never leak.
Common / Config	Cross-cutting concerns shared by all modules.	Consistency and reuse across the whole platform.

This structure means a change in one concern rarely forces a change in another: you can rename a database column without changing the API, add a validation rule without touching the database, or rewrite the security filter without touching business logic.

4. Configuration & Application Bootstrap

This layer wires the application together: it starts the Spring context, defines the security rules, binds configuration properties, configures CORS and Swagger, and seeds baseline data.

4.1 CivicDeskApplication — the Entry Point

File: `com/civicedesk/CivicDeskApplication.java`

- **WHAT:** The Spring Boot entry point. Its `main(String[] args)` calls `SpringApplication.run(CivicDeskApplication.class, args)`, which starts the Spring context, scans for components, launches the embedded Tomcat server, and runs any startup runners.
- **WHY:** Every Spring Boot application needs exactly one bootstrap class to trigger auto-configuration and component scanning.

- **HOW it connects:** Because the class sits in the root package `com.civicdesk`, its component scan covers every sub-package — `config`, `common`, and `module.iam.*`. After startup, the `DataSeeder` runner executes.

Annotation

- `@SpringBootApplication` — a meta-annotation combining three: `@Configuration` (the class may declare beans), `@EnableAutoConfiguration` (Spring configures itself based on the classpath — JPA, web, security), and `@ComponentScan` (discovers all `@Component/@Service/@Repository/@RestController/@Configuration` beans under `com.civicdesk`). It is what lets the entire module wire itself up with no XML.

4.2 SecurityConfig — the Heart of Authentication/Authorization

File: `com/civicdesk/config/SecurityConfig.java`

- **WHAT:** The single source of truth for how requests are secured. It defines the HTTP security filter chain (stateless JWT auth), the public-vs-protected URL rules, the placement of the custom JWT filter, the 401 entry point, and the password-encoder bean.
- **WHY:** It converts the application into a stateless, token-secured REST API: most endpoints require a valid JWT, a handful of auth/Swagger endpoints are public, and fine-grained role checks happen via `@PreAuthorize`.

The `filterChain(HttpSecurity http)` bean — step by step

1. `.cors(Customizer.withDefaults())` — enables CORS, looking up the `CorsConfigurationSource` bean defined in `CorsConfig`.
2. `.csrf(csrf -> csrf.disable())` — disables CSRF protection. This is correct for a stateless token API: there are no session cookies to protect, so CSRF tokens are unnecessary.
3. `.sessionManagement(s -> s.sessionCreationPolicy(STATELESS))` — the server keeps **no** HTTP session; every request must carry its own JWT.
4. `.authorizeHttpRequests(...)` — rules evaluated top-down:
 - `permitAll()` for `/iam/auth/register`, `/iam/auth/citizen/login`, `/iam/auth/staff/login`, `/iam/auth/setPassword` (these are how a user *obtains* a token).
 - `permitAll()` for Swagger paths (`/swagger-ui.html`, `/swagger-ui/**`, `/v3/api-docs`, `/v3/api-docs/**`).
 - `.anyRequest().authenticated()` — every other endpoint requires a valid authenticated principal.
5. `.exceptionHandling(...)` — a custom `AuthenticationEntryPoint` returns a clean HTTP **401** when a protected endpoint is hit without a valid token (instead of the default servlet error page).
6. `.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)` — inserts the custom `JwtAuthFilter` **before** Spring's username/password filter, so the request is already authenticated from its token by the time authorization runs.
7. `return http.build();` — builds the `SecurityFilterChain` bean.

The `passwordEncoder()` bean

Returns `new BCryptPasswordEncoder(10)` — a BCrypt encoder with cost factor 10. It is exposed as a `PasswordEncoder` bean and reused by `DataSeeder` (to hash the seeded admin) and the auth/user services (to hash on write and verify on login).

Annotations

- `@Configuration` — marks the class as a source of bean definitions.
- `@EnableMethodSecurity` — turns on annotation-based method security (`@PreAuthorize`, etc.). Without it, those annotations on controllers would be silently ignored. (Modern replacement for `@EnableGlobalMethodSecurity`.)
- `@Bean` — registers the two methods' return values as managed beans.

4.3 JwtConfig — Typed JWT Settings

File: `com/civicdesk/config/JwtConfig.java`

- **WHAT:** A typed configuration holder binding the `app.jwt.*` properties: `secret` (the HS256 signing secret) and `expiry` (token lifetime in milliseconds).
- **WHY:** Centralizes JWT settings into one strongly-typed object rather than scattering `@Value` lookups; components that mint/verify tokens inject this single bean.
- **HOW it connects:** Consumed by `JwtUtil` for signing/verifying tokens. The keys live in `application.properties` (`app.jwt.secret`, `app.jwt.expiry`).

Annotations

- `@Configuration` — registers it as a bean.
- `@ConfigurationProperties(prefix = "app.jwt")` — binds all properties under `app.jwt` to fields by name (type-safe, grouped configuration).

4.4 CorsConfig — Cross-Origin Policy

File: `com/civicdesk/config/CorsConfig.java`

- **WHAT:** Defines the CORS policy so a browser-based frontend on a different origin/port can call this backend.
- **WHY:** Browsers block cross-origin requests by default; this config explicitly allows the frontend origins, methods, headers, and credentials.
- **HOW:** A `@Value("${app.cors.allowed-origins:http://localhost:3000,http://localhost:5173}")` field supplies a comma-separated origin list (defaulting to common React/Vite dev ports). The `corsConfigurationSource()` bean builds a `CorsConfiguration` that allows those origins; methods `GET`, `POST`, `PUT`, `PATCH`, `DELETE`, `OPTIONS` (`OPTIONS` for preflight); all headers; exposes the `Authorization` response header (so the frontend can read a refreshed token); and enables credentials. It is registered for all paths (`/**`).
- **HOW it connects:** This bean is exactly what `SecurityConfig`'s `.cors(Customizer.withDefaults())` picks up.

4.5 OpenApiConfig — Swagger / API Documentation

File: `com/civicedesk/config/OpenApiConfig.java`

- **WHAT:** Configures the OpenAPI metadata (title, description, version) and registers a global **bearer (JWT)** security scheme so Swagger's "Authorize" button lets you paste a token and call secured endpoints.
- **HOW:** The `civicDeskOpenAPI()` bean returns an `OpenAPI` object with `.info(...)`, a global `SecurityRequirement` named `bearerAuth`, and a matching `SecurityScheme` of type HTTP / scheme `bearer` / format `JWT`.
- **HOW it connects:** The Swagger paths it enables are precisely the ones `SecurityConfig` marks `permitAll()`; the bearer scheme corresponds to the `Authorization: Bearer <token>` header validated by `JwtAuthFilter`.

4.6 DataSeeder — Startup Bootstrap Data

File: `com/civicedesk/config/DataSeeder.java` (implements `CommandLineRunner`)

- **WHAT:** A startup runner that seeds the six municipal departments and the very first admin account directly into the database — bypassing the API, since no admin exists yet to call protected endpoints.
- **WHY:** Bootstraps an empty database with the baseline data the system needs: departments to assign staff to, and a first administrator who can then create everyone else.
- **HOW:**
 - `run(String... args)` — the `CommandLineRunner` callback; calls `seedDepartments()` then `seedAdmin()`.
 - `seedDepartments()` — for each of the six fixed names (Infrastructure, Public Health, Licensing & Compliance, Citizen Services, Administration, Compliance & Audit), if it does not already exist, creates it with a generated id. Idempotent.
 - `nextDepartmentId()` — produces sequential ids `DPT01`, `DPT02`, ... by taking the max existing numeric suffix + 1 (zero-padded).
 - `seedAdmin()` — if no user with the admin email exists, builds a `User` with role `ADM`, status active, BCrypt-hashed password (from `app.admin.password`), `passwordSet = true`, no department; saves it; then records a `SEED_ADMIN` audit entry. Idempotent.
- **HOW it connects:** Runs automatically because `SpringApplication.run` executes all `CommandLineRunner` beans. Uses the `PasswordEncoder` bean from `SecurityConfig`, the IAM entities/enums/repositories, and `AuditService`. The seeded admin is the first principal able to authenticate and call the `@PreAuthorize`-protected endpoints.
- **Annotations:** `@Component` (discovered by component scan; its `run` executes at startup); `@Value` on the admin fields (injects `app.admin.*`, which are environment-overridable).

5. The Common Layer (Shared Building Blocks)

The `common` package holds module-agnostic infrastructure: a unified response envelope, a centralized exception handler with a family of domain exceptions, security/utility helpers, and a custom ID generator. These are shared across every module of the platform.

5.1 Response Envelopes (`common/response`)

5.1.1 ApiResponse

File: `common/response/ApiResponse.java`

- **WHAT:** A unified JSON response envelope with two fields, `message` and `data`, used for both success and error responses.
- **WHY:** Gives every endpoint a single, predictable shape. Null fields are suppressed, so the same class produces clean payloads for reads (data only), writes (message + optional data), and errors (message only).
- **HOW — factory methods:**
 - `ApiResponse.data(Object data)` — for GET success (data only).
 - `ApiResponse.of(String message, Object data)` — for POST/PUT success (message + optional data).
 - `ApiResponse.error(String message)` — for error responses (message only).
- **Annotation:** `@JsonInclude(JsonInclude.Include.NON_NULL)` (Jackson) — omits any null field during serialization, which is what makes `message/data` disappear when unset.
- **HOW it connects:** Returned by all IAM controllers and produced by `GlobalExceptionHandler` for errors.

5.1.2 ErrorResponse

File: `common/response/ErrorResponse.java`

- **WHAT:** A richer, path-aware error envelope carrying `status`, `message`, `path`, and `timestamp`; a static `of(...)` factory stamps the timestamp automatically.
- **WHY / connection:** Provided for cross-module consistency. The IAM `GlobalExceptionHandler` currently uses `ApiResponse` instead, so this type is available for other modules that prefer the richer shape.
- **Annotation:** `@JsonPropertyOrder({"status", "message", "path", "timestamp"})` — fixes the JSON field order for deterministic output.

5.1.3 PageResponse<T>

File: `common/response/PageResponse.java`

- **WHAT:** A lightweight, framework-neutral pagination envelope: `content`, `totalElements`, `totalPages`, `currentPage`.
- **WHY:** Decouples the public API from Spring Data's internal `Page` type, keeping the wire format stable and minimal.
- **HOW:** The key factory `static <S,T> PageResponse<T> from(Page<S> page, Function<S,T> mapper)` builds the envelope from a Spring Data `Page`, converting each entity to a response DTO via the mapper and copying the totals and current (zero-based) page number.
- **HOW it connects:** Used by the user-listing and audit-log endpoints; the mapper typically converts entity → DTO (e.g. `UserResponse::from`).

5.2 Exception Handling (`common/exception`)

5.2.1 GlobalExceptionHandler

File: `common/exception/GlobalExceptionHandler.java`

- **WHAT:** A centralized, application-wide handler that catches exceptions thrown anywhere in the controllers/services and converts each into an `ApiResponse.error(...)` body with a specific HTTP status.
- **WHY:** Keeps controllers free of `try/catch` and guarantees a consistent error envelope and status code for every error.
- **Annotations:** `@RestControllerAdvice` (global advice for all controllers; combines `@ControllerAdvice` + `@ResponseBody`), and one `@ExceptionHandler(Type.class)` per method.

Complete exception → HTTP status mapping

Exception	HTTP status	Meaning / when thrown
<code>BadCredentialsException</code> (app's own)	401 Unauthorized	Login failed — wrong email or password.
<code>TokenExpiredException</code>	401 Unauthorized	A JWT has expired.
<code>ForbiddenException</code>	403 Forbidden	Authenticated caller is not allowed to perform the action.
<code>AccessDeniedException</code> (Spring Security)	403 Forbidden	A <code>@PreAuthorize</code> role check failed (fixed message "Access denied").
<code>PasswordNotSetException</code>	403 Forbidden	Admin-created user who has not yet set a password tries to log in.
<code>BadRequestException</code>	400 Bad Request	Request is structurally valid but semantically invalid.
<code>AccountSuspendedException</code>	423 Locked	A suspended user tries to log in (credentials valid).
<code>AccountInactiveException</code>	403 Forbidden	An inactive/deactivated user tries to log in.

<code>DuplicateEmailException</code>	409 Conflict	Email already registered.
<code>ResourceNotFoundException</code>	404 Not Found	Requested resource does not exist.
<code>MethodArgumentNotValidException</code> (Spring)	400 Bad Request	A <code>@Valid</code> request body failed Bean Validation; the handler joins all field errors into one message (" <code>field: message, ...</code> ").

5.2.2 The Domain Exception Family

Nine custom exceptions all extend `RuntimeException` (unchecked) with a single message constructor. They give business code domain-meaningful signals while the handler owns the HTTP-status mapping (decoupling business logic from HTTP concerns).

- `ResourceNotFoundException`, `BadCredentialsException`, `DuplicateEmailException`, `ForbiddenException`, `TokenExpiredException`, `BadRequestException`, `AccountSuspendedException`, `AccountInactiveException`, `PasswordNotSetException`.

Note: The app's `BadCredentialsException` lives in `com.civicdesk.common.exception` and is distinct from Spring Security's class of the same name. The thoughtful split between `AccountSuspendedException` (423) and `AccountInactiveException` (403) lets clients react differently to a locked vs a merely inactive account.

5.3 Utilities (`common/util`)

5.3.1 JwtUtil — Token Authority

File: `common/util/JwtUtil.java` — covered in depth in **Section 10**. In summary, it is a `@Component` that issues and verifies HS256 JWTs using the JJWT library, injecting `JwtConfig` for the secret and expiry. Key methods: `generateToken(userId, role)`, `validateToken(token)`, `extractClaims/extractUserId/extractRole/extractExpiration`.

5.3.2 NationalIdUtil — PII Protection

File: `common/util/NationalIdUtil.java` (final, non-instantiable static helper)

- **WHAT/WHY:** Protects a citizen's national ID. The raw ID is never stored — only a one-way hash — and only the last four digits are ever displayed.
- **HOW:** `hash(nationalId)` computes a SHA-256 digest and hex-encodes it (returns null for blank input). `maskLast4(nationalId)` returns the value with all but the last four characters replaced by `*` (e.g. `*****1234`).

5.3.3 SecurityContextUtil — Reading the Current User

File: `common/util/SecurityContextUtil.java` (final, non-instantiable)

- **WHAT/WHY:** A clean, one-line way for service/controller code to read the currently authenticated caller without touching Spring Security internals. It reads exactly what `JwtAuthFilter` placed into the context.
- **HOW:** `getCurrentUserId()` returns the principal (the JWT `userId`); `getCurrentRole()` returns the first authority with the `ROLE_` prefix stripped.
- **HOW it connects:** Used by controllers to pass the caller's id/role into services, and by audit logging to record who acted.

5.3.4 ClientIpUtil — Resolving the Client IP

File: `common/util/ClientIpUtil.java` (final, non-instantiable)

- **WHAT/WHY:** Resolves the true client IP for audit records, accounting for proxies/load balancers and IPv6 loopback.
- **HOW:** `resolve(request)` prefers the first entry of the `X-Forwarded-For` header, falling back to `getRemoteAddr()`; it normalizes IPv6 loopback (`::1`) to `127.0.0.1` and returns `"UNKNOWN"` when nothing is available.
- **HOW it connects:** Controllers call it and pass the result into `AuditService.log(...)`.

5.4 Custom ID Generation (`common/id`)

NumericStringSequenceGenerator

File: `common/id/NumericStringSequenceGenerator.java` (extends Hibernate's `SequenceStyleGenerator`)

- **WHAT:** A custom Hibernate ID generator that produces sequential numeric ids (`10000001`, `10000002`, ...) from a database sequence but stores them in a `String` primary-key column.
- **WHY:** The whole codebase treats user/audit ids as opaque `Strings` (the JWT `userId` claim, the security principal, repository keys, DTOs). Keeping the id a `String` avoids a type change rippling through every layer, while giving human-friendly running numbers instead of UUIDs. It works on both H2 (native sequences, used in tests) and MySQL (Hibernate emulates with a backing table).
- **HOW:** `configure(...)` tells the parent generator the type is `Long/BIGINT` (so it can drive the sequence) even though the entity field is a `String`; `generate(...)` fetches the next numeric value and converts it with `String.valueOf(...)` before it is assigned to the `@Id` field.
- **HOW it connects:** Referenced by the `User` and `AuditLog` entity `@Id` mappings; the strings it mints become the JWT `userId` and the Spring Security principal.

6. The Data Layer: Entities, Enums, Repositories & Schema

6.1 Cross-Cutting Persistence Design

- **Naming strategy:** the app uses `PhysicalNamingStrategyStandardImpl`, so Hibernate keeps identifiers as-is (camelCase columns like `userId`, `passwordHash`, `createdAt`) rather than converting to snake_case.
- **ID generation:** `User` and `AuditLog` use the custom `NumericStringSequenceGenerator` (sequential numeric strings from `10000001`); `Department` uses hand-assigned `DPT01`–style keys.
- **Enums stored as strings:** entity columns `role`, `status`, `action`, `module` are plain `String` (no `@Enumerated`). The enums exist for application-layer validation/normalization; the database sees raw strings, which keeps the values stable and module-shareable.

6.2 Entity: User

File: `module/iam/entity/User.java`

- **WHAT/WHY:** The JPA entity mapped to the `users` table — the core identity record for every account (citizen, staff, admin) and the single source of truth for authentication, authorization, org placement, and lifecycle.
- **Class annotations:** `@Entity` (JPA-managed persistent class), `@Table(name="users", indexes={...})` declaring three indexes — `idx_users_role`, `idx_users_status`, and the composite `idx_users_role_status` (supporting the common combined filter).

Field	Column / Type	Constraints & notes
<code>userId</code>	<code>userId</code> , <code>String(36)</code>	PK, not null, not updatable; generated sequential numeric string.
<code>name</code>	<code>name</code> , <code>String(100)</code>	not null.
<code>email</code>	<code>email</code> , <code>String(150)</code>	not null, unique — the login identifier.
<code>passwordHash</code>	<code>passwordHash</code> , <code>String(255)</code>	nullable; null until the owner sets a password (BCrypt).
<code>passwordSet</code>	<code>isPasswordSet</code> , <code>boolean</code>	not null, default false — “has the owner set a password?”
<code>phone</code>	<code>phone</code> , <code>String(15)</code>	nullable.
<code>role</code>	<code>role</code> , <code>String(30)</code>	not null; one of the <code>Role</code> enum names.
<code>departmentId</code>	<code>departmentId</code> , <code>String(36)</code>	nullable; informal reference to <code>Department</code> .

<code>status</code>	status, String(20)	not null, default "A" (active).
<code>createdAt</code>	createdAt, LocalDateTime	not updatable; auto-set on insert.
<code>updatedAt</code>	updatedAt, LocalDateTime	auto-updated on every change.

Field/ORM annotation glossary

- `@Id` — marks the primary-key field.
- `@GeneratedValue(generator="...")` — delegates id creation to a named generator.
- `@GenericGenerator(... NumericStringSequenceGenerator ...)` — a Hibernate hook pointing at the custom generator (with `@Parameter` sequence settings).
- `@Column(...)` — maps a field to a column and declares length/nullability/uniqueness.
- `@CreationTimestamp` / `@UpdateTimestamp` — Hibernate auto-populates these at insert / on every update.
- `@Index` (inside `@Table`) — declares a database index for faster filtering.

6.3 Entity: AuditLog

File: `module/iam/entity/AuditLog.java`

- **WHAT/WHY:** Maps the `audit_log` table — an append-only trail of security/lifecycle events for compliance and forensics (who did what, in which module, from which IP, and when).
- **HOW:** Insert-only; `timestamp` is auto-set via `@CreationTimestamp` (no update timestamp since rows are immutable). Same numeric-string id pattern (`audit_id_seq`).
- **Annotations:** `@Entity`, `@Table(name="audit_log", indexes={...})` with four indexes (`idx_audit_userId`, `idx_audit_action`, `idx_audit_module`, `idx_audit_timestamp`) supporting filtering and time-ordered listing.

Field	Column / Type	Notes
<code>auditId</code>	auditId, String(36)	PK, generated numeric string.
<code>userId</code>	userId, String(36)	not null — the actor.
<code>action</code>	action, String(50)	not null — an <code>AuditAction</code> name (uppercase).
<code>module</code>	module, String(50)	not null — an <code>AuditModule</code> name (uppercase).
<code>ipAddress</code>	ipAddress, String(45)	nullable; length 45 fits IPv6.

timestamp	timestamp, LocalDateTime	not updatable; auto-set at insert.
-----------	--------------------------	------------------------------------

6.4 Entity: Department

File: `module/iam/entity/Department.java`

- **WHAT/WHY:** Maps the `departments` table — the organizational units that group staff and track the heading supervisor; provides a stable, human-readable code.
- **HOW:** The simplest entity. `departmentId` (String(10)) is hand-assigned (e.g. `DPT01`) — no `@GeneratedValue`. `name` (String(100)) is not null and unique. `departmentSupervisorId` (String(36), nullable) informally references the heading `User`. A convenience constructor `Department(String name)` exists.
- **Annotations:** `@Entity`, `@Table(name="departments")`, `@Id`, `@Column`.

6.5 Enums

Enum	Values	Purpose
<code>Role</code>	CIT (Citizen), FO (Field Officer), DS (Department Supervisor), ENG (Engineer), CO (Coordinator/Compliance Officer), ADM (Admin)	Valid user roles; stored as the enum name in <code>User.role</code> ; the <code>@PreAuthorize</code> and <code>users.role</code> contract.
<code>UserStatus</code>	ACT("A") Active, INA("I") Inactive, SUS("S") Suspended	Maps friendly names to single-char codes stored in <code>User.status</code> ; <code>normalize(input)</code> canonicalizes varied inputs (e.g. "active", "ACT", "A" → "A").
<code>AuditAction</code>	REGISTER, LOGIN, LOGOUT, CREATE_USER, UPDATE_STATUS, SET_PASSWORD, SEED_ADMIN	Canonical auditable actions; name stored in <code>AuditLog.action</code> .
<code>AuditModule</code>	IAM, SERVICE_REQUEST, GRIEVANCE, PERMIT, WORKS	Which functional module emitted the event; name stored in <code>AuditLog.module</code> .

`UserStatus` carries a `label` field with a getter, and its `normalize` method is used at login, on status updates, and when validating a status filter.

6.6 Repositories (`module/iam/repository`)

Spring Data JPA repositories are *interfaces*: you declare a method and Spring generates the SQL implementation at runtime. All three are annotated `@Repository` and extend

`JpaRepository<Entity, String>` (CRUD + paging). Two also extend `JpaSpecificationExecutor` for dynamic queries.

Repository	Notable methods	Purpose
<code>UserRepository</code> (+ <code>JpaSpecificationExecutor<User></code>)	<code>findByEmail</code> , <code>existsByEmail</code> , <code>findByDepartmentId(...)</code> , <code>findByDepartmentIdAndStatus(...)</code> , <code>findByRole(...)</code>	Login lookup, duplicate check, and paginated filtered listings. The specification executor enables <code>findAll(Specification, Pageable)</code> to combine filters without an explosion of method variants.
<code>AuditLogRepository</code> (+ <code>JpaSpecificationExecutor<AuditLog></code>)	<code>findById(...)</code> , <code>findByModule(...)</code> , <code>findAllOrderByTimestampDesc(...)</code>	Per-user / per-module queries and newest-first listing; specifications drive combined <code>userId/action/module</code> filtering.
<code>DepartmentRepository</code>	<code>existsByName</code> , <code>findByName</code>	Uniqueness check and lookup by name (no specification support needed).

JPA / Hibernate in one paragraph: JPA is the specification; Hibernate is the implementation. An *entity* is a Java object mapped to a table; the *EntityManager* tracks entities and flushes changes as SQL. *Derived query methods* (like `findByEmail`) are parsed from the method name into SQL. *Specifications* build type-safe dynamic **WHERE** clauses with the JPA Criteria API, so optional filters can be combined with `.and(...)` at run time.

6.7 Specifications (`module/iam/repository/spec`)

- `UserSpecifications` — a final utility class of composable `Specification<User>` predicates: `hasRole`, `hasStatus`, `inDepartment` (each a single-column equality). Callers `.and(...)` only the filters supplied.
- `AuditLogSpecifications` — composable `Specification<AuditLog>` predicates: `hasUserId` (verbatim), `hasAction` and `hasModule` (each trims and upper-cases the argument, because actions/modules are stored uppercase).

Both are passed to the repositories' `findAll(Specification, Pageable)`, letting the user-listing and audit-log endpoints support any combination of optional filters with one query path.

6.8 Database Schema (`src/main/resources/db/civicsdesk_iam.sql`)

- **WHAT/WHY:** A hand-maintained MySQL 8 schema mirroring the three entities, plus optional foreign keys, seed data, and demonstration DML. It documents the schema and lets a DBA stand up the database by hand (at runtime the app does this automatically via `ddl-auto=update` and `DataSeeder`).
- **Tables:** `departments`, `users` (with the three indexes), and `audit_log` (with four indexes), all InnoDB / utf8mb4.
- **Optional foreign keys** (commented out, since Hibernate creates none): `users.departmentId` → `departments`, `departments.departmentSupervisorId` → `users` (defined after both tables to avoid a cyclic dependency), and `audit_log.userId` → `users`.
- **Seed data:** the six departments, the system admin (`10000001`), and a `SEED_ADMIN` audit row — mirroring exactly what `DataSeeder` creates.
- **Demonstration DML:** illustrative INSERT/UPDATE/SELECT/DROP statements (e.g. assigning supervisors, suspending a user, counting users by role).

7. Data Transfer Objects (DTOs)

7.1 Why DTOs Instead of Exposing Entities

DTOs form the contract between the API and its clients. They exist separately from JPA entities for three reasons:

- **Security:** entities carry sensitive/internal fields (e.g. `passwordHash`, internal flags). Returning them risks leaking secrets, and binding requests straight onto them risks “mass-assignment” attacks (a client setting a field it shouldn’t, like its own role). `UserResponse` deliberately **omits the password hash**.
- **Decoupling:** the wire format can evolve independently of the database. Renaming a column does not break the API, and vice versa.
- **Validation & performance:** request DTOs are the boundary where input is validated (rejecting malformed data with 400 before it reaches business logic), and they carry only the fields actually needed (no over-fetching of entity graphs).

Implementation note: these DTOs are plain POJOs with hand-written getters/setters and explicit constructors — Lombok is *not* used in this codebase. (Lombok’s `@Data/@Builder` etc. are explained generally in the annotation reference for completeness.) Response DTOs each provide a static `from(entity)` factory for entity→DTO mapping.

7.2 Request DTOs (`dto/request`)

DTO	Fields	Key validations & purpose
<code>RegisterRequest</code>	name, email, password, phone, dateOfBirth, gender, nationalId,	Citizen self-registration. <code>@NotBlank/@Size</code> on name/email; <code>@Email</code> ; password <code>@Size(min=8,max=72)</code> (72 = BCrypt’s effective byte limit); phone <code>@Pattern("^[6-9][0-9]{9}\$")</code>

	address, ward, zone	(Indian mobile). Optional profile fields use format <code>@Patterns</code> (date <code>YYYY-MM-DD</code> , gender <code>MALE/FEMALE/OTHER</code> , alphanumeric national id).
<code>CitizenLoginRequest</code>	email, password	Citizen login. <code>@NotBlank</code> + <code>@Email</code> on email; <code>@NotBlank</code> on password.
<code>StaffLoginRequest</code>	email, password	Staff login. Structurally identical to citizen login but kept a separate type so the two flows can diverge.
<code>SetPasswordRequest</code>	email, newPassword	First-time password setup for admin-created staff. <code>@Email</code> on email; newPassword <code>@Size(min=8,max=72)</code> .
<code>CreateUserRequest</code>	name, email, phone, role, departmentId	Admin-driven staff creation (no password field — staff set their own later). <code>role</code> restricted via <code>@Pattern(DS FO ENG CO)</code> ; optional <code>departmentId</code> .
<code>UpdateUserStatusRequest</code>	status	Status change. <code>@Pattern</code> accepts case-insensitive short/long forms (A/ACT/ACTIVE, I/INA/INACTIVE, S/SUS/SUSPENDED).

Validation annotations explained

- `@NotBlank` — the string must be present and not just whitespace.
- `@NotNull` — the value must not be null (used for non-string fields).
- `@Email` — the string must be a syntactically valid email.
- `@Size(min, max)` — bounds string length (e.g. password 8–72; name ≤100).
- `@Pattern(regex=...)` — the string must match a regular expression (phone, role whitelist, date format, status whitelist).

Using `@Pattern` whitelists for `role` and `status` gives clean, human-readable 400 messages instead of cryptic deserialization errors. Validation only fires because the controllers annotate the body with `@Valid`.

7.3 Response DTOs (`dto/response`)

DTO	Fields	Purpose / mapping
<code>AuthResponse</code>	token, userId, role, expiresIn	Returned on successful login. Carries the JWT and minimal identity info; <code>expiresIn</code> is the token lifetime in seconds. Assembled in the auth service (no <code>from</code> factory).

<code>UserResponse</code>	userId, name, email, phone, role, departmentId, status, createdAt	The safe projection of <code>User</code> — never includes the password hash . Built with <code>UserResponse.from(user)</code> .
<code>DepartmentResponse</code>	departmentId, name, departmentSupervisorId	Safe projection of <code>Department</code> ; built with <code>from(d)</code> .
<code>AuditLogResponse</code>	auditId, userId, action, module, ipAddress, timestamp	Audit-trail entry exposed to admins/compliance; built with <code>from(log)</code> .

8. The Service Layer (Business Logic)

8.1 Why a Service Layer Exists

The service layer sits between the web layer (controllers) and the persistence layer (repositories). Its purpose is **separation of concerns**:

- **Business-logic isolation** — rules like “an Admin may only create a Department Supervisor”, password validation, status normalization, and role-based filtering live here, not in controllers, making them reusable and independently testable.
- **Transaction boundaries** — multi-step writes are wrapped in `@Transactional` so they commit or roll back atomically.
- **Abstraction via interfaces** — each service is an interface plus a `@Service` implementation, so controllers depend on the abstraction and tests can mock it.

Each service follows the pattern: an interface (`AuthService`, `UserService`, `AuditService`, `DepartmentService`) and an implementation (`...Impl`). All use constructor injection with `final` fields (no field `@Autowired`).

8.2 AuthServiceImpl

Files: `service/AuthService.java` (interface), `service/AuthServiceImpl.java`

WHAT/WHY: Implements registration, citizen/staff login, and first-time password setup. It centralizes all credential verification, account-status gating, password hashing, and token issuance so controllers stay thin. Dependencies: `UserRepository`, `PasswordEncoder`, `JwtUtil`, `AuditService`.

Methods, step by step

- **register(req, ip)** — `@Transactional`. Rejects a duplicate email (`DuplicateEmailException`); builds a citizen `User`; BCrypt-hashes the password; sets role `CIT`, active status, `passwordSet=true`; saves; writes a `REGISTER` audit entry.
- **citizenLogin(req, ip)** — calls the shared `authenticate(...)`; if the user is *not* a citizen, throws `ForbiddenException("Please use the staff portal")`; otherwise issues a token.

- **staffLogin(req, ip)** — calls `authenticate(...)`; if the user is a citizen, throws `ForbiddenException("Please use the citizen portal")`; otherwise issues a token.
- **authenticate(email, password)** (private) — the shared credential + status gate: looks up by email (else `BadCredentialsException`); if `!passwordSet` throws `PasswordNotSetException`; verifies the password with `passwordEncoder.matches(...)` (else `BadCredentialsException` with the same generic message to prevent user enumeration); normalizes status and throws `AccountSuspendedException` (suspended) or `AccountInactiveException` (not active).
- **setPassword(req)** — `@Transactional`. Looks up the user (else `ResourceNotFoundException`); if a password is already set throws `ForbiddenException` (one-time only); enforces minimum length; BCrypt-hashes, sets `passwordSet=true`, saves; writes a `SET_PASSWORD` audit entry.
- **issueToken(user, ip)** (private) — generates a JWT via `jwtUtil.generateToken(userId, role)`; writes a `LOGIN` audit entry; returns an `AuthResponse` with the token, id, role, and expiry in seconds.

Connections: `UserRepository`, `PasswordEncoder`, `JwtUtil`, `AuditService`; the `Role/UserStatus/AuditAction` enums; and the domain exceptions. **Annotations:** `@Service`; `@Transactional` on the two write methods; `@Value("${app.jwt.expiry}")` for the expiry.

8.3 UserServiceImpl

Files: `service/UserService.java`, `service/UserServiceImpl.java`

WHAT/WHY: Implements user lookup, role-based creation, status updates, and paged filtered listing — enforcing the organizational hierarchy and scoping data visibility by department. Dependencies: `UserRepository`, `DepartmentRepository`.

Methods

- **getId(userId)** — returns `UserResponse.from(user)` or throws `ResourceNotFoundException`.
- **createUser(req, callerRole, callerUserId)** — `@Transactional`. After a duplicate-email check, applies the role rules:
 - **Caller is ADM** → may only create a `DS`; `departmentId` is required and must exist (the created supervisor is then back-linked as the department's `departmentSupervisorId`).
 - **Caller is DS** → may only create `FO/ENG/CO`, and the new user inherits the supervisor's own department (the caller cannot pick an arbitrary one).
 - **Any other role** → `ForbiddenException`.

New staff are saved with `passwordHash=null` and `passwordSet=false` (they set a password on first login).

- **updateStatus(userId, status)** — normalizes the status with `UserStatus.normalize(...)` and saves.
- **getUsers(callerRole, callerUserId, role, status, departmentId, page, size)** — builds a dynamic `Specification`: an `ADM` may filter by role/status/department and sees all users; a `DS` is force-scoped to *active* users in *their own* department. Executes `findAll(spec, pageable)` and returns a `PageResponse<UserResponse>`.

Annotations: `@Service`; `@Transactional` on `createUser` (it performs two writes — the new user and the department back-link — that must be atomic).

8.4 AuditServiceImpl

Files: `service/AuditService.java`, `service/AuditServiceImpl.java`

- **WHAT/WHY:** The single point for the audit trail — writes entries and returns paged, filtered, newest-first history. Reused by the auth flows to record REGISTER/LOGIN/SET_PASSWORD. Dependency: `AuditLogRepository`.
- `log(userId, action, module, ip)` — constructs an `AuditLog` with the given fields and saves it (timestamp auto-populated).
- `getAll(userId, action, module, page, size)` — composes `AuditLogSpecifications` for the non-blank filters, sorts by `timestamp` descending, and returns a `PageResponse<AuditLogResponse>`.
- **Annotation:** `@Service` (no `@Transactional` — each method is a single repository operation).

8.5 DepartmentServiceImpl

Files: `service/DepartmentService.java`, `service/DepartmentServiceImpl.java`

- **WHAT/WHY:** A simple read service returning all departments sorted by name (used when an Admin picks a department while creating a Supervisor). Dependency: `DepartmentRepository`.
- `getAll()` — `findAll(Sort.by("name"))`, mapped to `DepartmentResponse` via the static factory.
- **Annotation:** `@Service`.

8.6 Service-Layer Summary

Aspect	Detail
Bean stereotype	All impls are <code>@Service</code> with constructor-injected <code>final</code> dependencies.
<code>@Transactional</code>	Only on multi-write methods: <code>register</code> , <code>setPassword</code> , <code>createUser</code> .
Password security	Exclusively through <code>PasswordEncoder.encode/matches</code> ; uniform “Invalid email or password” on login to prevent enumeration.
JWT	Generated only in <code>AuthServiceImpl.issueToken</code> ; expiry from <code>app.jwt.expiry</code> (ms), surfaced in seconds.
Audit	Centralized in <code>AuditService.log</code> ; auth flows record REGISTER/LOGIN/SET_PASSWORD.

Role hierarchy	ADM → creates DS (valid department, back-links supervisor). DS → creates FO/ENG/CO (own department). Others forbidden.
Data scoping	ADM sees all; DS restricted to own department & active users only.

9. The Controller Layer (REST API)

9.1 Role of Controllers & the “Thin Controller” Principle

Controllers are the HTTP front door. In a REST API they map URLs and HTTP verbs to Java methods, read request bodies/parameters, and serialize responses to JSON. In this codebase every controller method does only orchestration:

1. Read the caller’s identity/role from the security context (`SecurityContextUtil`).
2. Resolve the client IP (`ClientIpUtil`) when auditing.
3. Delegate the real work to a service method.
4. Optionally fire an audit log.
5. Wrap the result in `ApiResponse` / `ResponseEntity`.

There is **no business logic, persistence, or mapping** in controllers — methods are typically 3–6 lines. This keeps them focused on HTTP concerns and makes the business layer independently testable. `DepartmentController` is the clearest example: a single one-line delegation.

9.2 Common Controller Annotations

- `@RestController` — a controller whose return values are written directly to the HTTP body as JSON (combines `@Controller` + `@ResponseBody`).
- `@RequestMapping("/path")` — the shared base path for all methods in the class.
- `@GetMapping` / `@PostMapping` / `@PutMapping` — map a method to HTTP GET / POST / PUT at a sub-path.
- `@PathVariable` — binds a `{placeholder}` URL segment to a parameter.
- `@RequestParam` — binds a query-string parameter (supports `required=false` and `defaultValue`).
- `@RequestBody` — deserializes the JSON body into a DTO.
- `@Valid` — triggers Bean Validation on the DTO before the method runs.
- `@PreAuthorize("...")` — a method-level security gate using SpEL (`hasRole('ADM')`, `hasAnyRole('ADM', 'DS')`). Spring matches the `ROLE_`-prefixed authority the JWT filter installs.
- **Constructor injection** — dependencies are `final` fields set through the constructor (no `@Autowired` needed for a single constructor).

9.3 AuthController

File: `controller/AuthController.java` — base path `/iam/auth`. The public entry point into authentication. Dependencies: `AuthService`, `AuditService`.

Method	Verb + path	Access	Body / input	Result
<code>register</code>	POST <code>/iam/auth/register</code>	Public	<code>@Valid RegisterRequest + HttpServletRequest</code>	201 + "Registration successful"
<code>citizenLogin</code>	POST <code>/iam/auth/citizen/login</code>	Public	<code>@Valid CitizenLoginRequest</code>	200 + <code>AuthResponse</code> (JWT)
<code>staffLogin</code>	POST <code>/iam/auth/staff/login</code>	Public	<code>@Valid StaffLoginRequest</code>	200 + <code>AuthResponse</code> (JWT)
<code>setPassword</code>	POST <code>/iam/auth/setPassword</code>	Public	<code>@Valid SetPasswordRequest</code>	200 + "Password set successfully"
<code>logout</code>	POST <code>/iam/auth/logout</code>	Authenticated	<code>HttpServletRequest</code>	200; records a <code>LOGOUT</code> audit entry

Login/register audit logging is delegated into the service (the IP is passed through); `logout` audits directly in the controller. Because the API is stateless, logout does not invalidate the token server-side — it records the event and the client discards the token.

9.4 UserController

File: `controller/UserController.java` — base path `/iam/users`. Dependencies: `UserService`, `AuditService`.

Method	Verb + path	Access	Notes
<code>getMe</code>	GET <code>/iam/users/me</code>	Any authenticated user	Returns the caller's own profile (id from the security context).
<code>createUser</code>	POST <code>/iam/users</code>	<code>hasAnyRole('ADM', 'DS')</code>	201. Passes caller role + id into the service so it can enforce the

			creation rules; audits <code>CREATE_USER</code> .
<code>getUsers</code>	GET <code>/iam/users</code>	<code>hasAnyRole('ADM', 'DS')</code>	Optional <code>role</code> , <code>status</code> , <code>departmentId</code> , <code>page</code> (0), <code>size</code> (10). Returns a paged list.
<code>updateStatus</code>	PUT <code>/iam/users/{id}/status</code>	<code>hasRole('ADM')</code>	<code>@PathVariable id + @Valid UpdateUserStatusRequest</code> ; audits <code>UPDATE_STATUS</code> .

9.5 AuditLogController

File: `controller/AuditLogController.java` — base path `/iam/auditLogs`. A single read-only endpoint.

Method	Verb + path	Access	Notes
<code>getAuditLogs</code>	GET <code>/iam/auditLogs</code>	<code>hasAnyRole('ADM', 'CO')</code>	Newest-first. Optional <code>userId</code> , <code>action</code> , <code>module</code> , <code>page</code> (0), <code>size</code> (20). This controller only <i>reads</i> the trail; writing happens elsewhere via <code>auditService.log</code> .

9.6 DepartmentController

File: `controller/DepartmentController.java` — base path `/iam/departments`.

Method	Verb + path	Access	Notes
<code>getDepartments</code>	GET <code>/iam/departments</code>	<code>hasAnyRole('ADM', 'DS')</code>	Lists all departments (e.g. to pick a valid <code>departmentId</code> when creating a supervisor). The thinnest controller — a single delegation.

10. Security & JWT Authentication (In Depth)

This is the core of an IAM module. This section explains JWT conceptually, then walks through every line of the code that issues, transmits, validates, and authorizes with tokens.

10.1 What Is a JWT?

A **JSON Web Token (JWT)** is a compact, URL-safe, self-contained credential. “Self-contained” means the token itself carries the identity facts (claims) the server needs — the server does not look anything up in a session store to know who the caller is. In CivicDesk, a successful login produces a JWT, and the client sends it back on every request as the header `Authorization: Bearer <token>`.

10.2 The Three Parts of a JWT

A JWT is three Base64URL segments joined by dots: `header.payload.signature`.

```
eyJhbGciOiJIUzI1NiJ9 . eyJ1c2VySWQiOiIzMDAwMDAwMSIsInJvbGUiOiJBRE0iLCJpYXQiOi4uLn0 .  
3x9Kk2...signature  
└─── HEADER ───┬─────────── PAYLOAD (claims) ───────────┬───  
SIGNATURE ───┘
```

- **Header** — JSON metadata about the token, notably the signing algorithm. JWT writes `{"alg": "HS256"}`.
- **Payload (claims)** — the data. This codebase puts:
 - `userId` — the user’s id → becomes the Spring Security **principal**.
 - `role` — the user’s role → becomes the `ROLE_<role>` **authority**.
 - `iat` (issued-at) and `exp` (expiration, 30 minutes out).

Claims are only Base64URL-**encoded**, not encrypted — anyone can read them, so no secrets are placed there.

- **Signature** — proves the token was issued by this server and has not been altered.

10.3 How HS256 Signing Works

HS256 = **HMAC with SHA-256**, a *symmetric* scheme. The server computes:

```
signature = HMAC_SHA256( base64url(header) + "." + base64url(payload), SECRET )
```

using the single shared secret from `app.jwt.secret`. On every request, validation recomputes the HMAC over the received `header.payload` with the same secret and checks it matches the token’s signature. If even one byte of the payload changed (e.g. an attacker editing `role` to gain admin), the recomputed signature would not match and verification fails. Because the same secret signs and verifies, it must stay server-side only — HS256 also requires the secret to be at least 256 bits / 32 bytes, which is why the configured secret is “minimum_32_characters”.

10.4 Token Generation & Verification — JwtUtil

File: `common/util/JwtUtil.java` (`@Component`, injects `JwtConfig`)

- **Signing key** — `getKey()` derives an HMAC-SHA key from the UTF-8 bytes of the secret via `Keys.hmacShaKeyFor(...)`.
- **`generateToken(userId, role)`** — builds the token: `.claim("userId", userId), .claim("role", role), .issuedAt(now), .expiration(now + expiry), .signWith(getKey())` (HS256 inferred), `.compact()`.
- **`validateToken(token)`** — parses and verifies signature/structure/expiry; returns `true` on success, `false` on any exception (bad signature, malformed, expired).
- **`extractClaims` / `extractUserId` / `extractRole` / `extractExpiration`** — verify and read the payload; unlike `validateToken`, these *throw* on an invalid/expired token. `extractExpiration` feeds the sliding-expiry refresh.

10.5 The Per-Request Gatekeeper — `JwtAuthFilter`

File: `module/iam/security/JwtAuthFilter.java` (extends `OncePerRequestFilter`)

Why `OncePerRequestFilter`? It guarantees the filter runs exactly once per HTTP request even across internal forwards/async dispatches — you do not want to re-parse the token or re-populate the context multiple times.

`doFilterInternal(req, res, chain)` — step by step

1. Read the `Authorization` header.
2. If it is missing or does not start with `"Bearer "`, **pass through unauthenticated** (`chain.doFilter`) — this is what lets the `permitAll()` endpoints work without a token; protected routes are rejected later by the 401 entry point.
3. Strip the `"Bearer "` prefix to get the raw token.
4. If `!jwtUtil.validateToken(token)`, set status **401** and stop — an invalid/expired/tampered token is rejected immediately.
5. Extract `role` and `userId` from the verified token.
6. Build a `UsernamePasswordAuthenticationToken` with `principal = userId`, `credentials = null`, and a single authority `"ROLE_" + role` (the `ROLE_` prefix is what `hasRole(...)` expects). The 3-argument constructor marks it authenticated.
7. Store it in `SecurityContextHolder` — the caller is now authenticated for the rest of the request.
8. **Sliding-expiry refresh:** compute seconds remaining from the `exp` claim; if `≤ 600` (10 minutes), mint a fresh 30-minute token and return it on the response `Authorization` header, plus `Access-Control-Expose-Headers: Authorization` so browsers let JavaScript read it. Active users stay logged in without re-authenticating, with no server-side state.
9. Continue the chain to the controller.

Token state	Behavior
Missing / not "Bearer "	Pass through unauthenticated; protected routes 401 via the entry point.
Present but invalid / expired / tampered	Immediate 401; chain stops.

Valid	Authenticate, optionally refresh, continue.
-------	---

Annotation: `@Component` (so `SecurityConfig` can inject it). It is wired explicitly with `addFilterBefore(...)` rather than auto-registered.

10.6 Why JWT Is Stateless (and Why It Matters Here)

Because all identity data and a tamper-proof signature travel *inside* the token, the server keeps no session record. That is why `SecurityConfig` uses `SessionCreationPolicy.STATELESS` and `JwtAuthFilter` rebuilds the `Authentication` from scratch on every request. Any server instance holding the shared secret can verify any token — no sticky sessions and no shared session store, which makes horizontal scaling trivial.

10.7 The Authorization Process (RBAC)

- The filter installs a `ROLE_<role>` authority from the token's `role` claim.
- `@EnableMethodSecurity` (in `SecurityConfig`) activates `@PreAuthorize` on controller methods.
- Each protected endpoint declares its rule: `hasRole('ADM')` (status updates), `hasAnyRole('ADM', 'DS')` (user management, departments), `hasAnyRole('ADM', 'CO')` (audit logs).
- If the role does not satisfy the rule, Spring throws `AccessDeniedException`, which `GlobalExceptionHandler` turns into a clean **403**.
- Finer business rules that a single role check cannot express (e.g. “a DS may only see their own department”) are enforced in the service layer using the caller's id/role.

10.8 Advantages & Limitations of This Design

Advantages	Limitations
No session store → easy horizontal scaling.	Hard to revoke before expiry — a leaked token is valid until <code>exp</code> (up to 30 min); there is no server-side blacklist.
Decoupled & reusable — every module trusts the same <code>userId/role</code> claims (<code>JwtUtil</code> is shared across teams).	Symmetric secret — the same secret signs and verifies, so a leak lets attackers forge tokens (RS256/ES256 would avoid sharing the signing key).
Fast authorization — the role comes straight from the token, no per-request DB hit.	Payload is readable (encoded, not encrypted) — never store sensitive data in claims.

Built-in expiry + sliding refresh keeps active users logged in smoothly.

Sliding refresh can extend an actively-used token indefinitely — there is no absolute session cap.

11. End-to-End Flows

11.1 Generic Request Flow (API call → Controller → Service → DB → Response)

1. Client sends an HTTP request with `Authorization: Bearer <jwt>`
2. CORS filter → applies the allowed-origins policy (`CorsConfig`)
3. `JwtAuthFilter` → validates the token, sets the `SecurityContext` (principal + `ROLE_x`)
4. Authorization check → URL rule (`authorizeHttpRequests`) + `@PreAuthorize` role
5. Controller method runs → reads body/params (`@RequestBody/@RequestParam`), `@Valid`
6. Service method audits → business rules, calls repositories, hashes/issues tokens,
7. Repository (JPA) → Hibernate generates SQL against MySQL/H2
8. Entity ↔ DTO → service maps entities to response DTOs (e.g. `UserResponse.from()`)
9. Controller → wraps the DTO in `ApiResponse` → Jackson serializes to JSON
10. (any exception) → `GlobalExceptionHandler` maps it to `{message}` + the right status
11. Response returned → standard envelope `{ success?, statusCode?, message, data, timestamp }`

11.2 Authentication Flow (Login → Token)

```
POST /iam/auth/citizen/login { email, password }
|
▼
AuthController.citizenLogin → AuthServiceImpl.citizenLogin
|   authenticate(): findByEmail → check passwordSet → passwordEncoder.matches →
status gate
|   portal check: must be a CIT (else 403 "use the staff portal")
|
▼
issueToken(): JwtUtil.generateToken(userId, role) + audit LOGIN
|
▼
ApiResponse { token, userId, role, expiresIn=1800 } → 200 OK
|
▼
Client stores the token and sends it on every subsequent request.
```


11.3 Staff Onboarding Flow (Set-Password on First Login)

Admin creates staff (no password)	→ <code>isPasswordSet = false</code>
Staff POST <code>/iam/auth/staff/login</code> logging in"	→ 403 "Please set your password before logging in"
Staff POST <code>/iam/auth/setPassword</code> { email, newPassword }	→ validates length, BCrypt-hashes, sets <code>isPasswordSet = true</code> (200)
Staff POST <code>/iam/auth/staff/login</code> again (setPassword works ONCE per account; afterwards → 403, use a reset flow)	→ 200 + JWT

11.4 Data Flow Between Layers

- **Inbound:** JSON → **request DTO** (validated by `@Valid`) → service → **entity** → repository → database.
- **Outbound:** database → repository → **entity** → service maps to **response DTO** (`from(...)`) → wrapped in `ApiResponse/PageResponse` → JSON.
- Entities never cross the controller boundary, so the database shape never leaks to clients.

12. Complete Annotation Reference

Every annotation used in the project, grouped by family, with what it does and why.

12.1 Core Spring / Stereotypes

Annotation	What it does & why
<code>@SpringBootApplication</code>	Bootstraps the app (config + auto-configuration + component scan). One per application.
<code>@Configuration</code>	Marks a class that declares <code>@Bean</code> definitions.
<code>@Bean</code>	Registers a method's return value as a Spring-managed bean.
<code>@Component</code>	Generic stereotype for a Spring-managed bean (used for <code>DataSeeder</code> , <code>JwtUtil</code> , <code>JwtAuthFilter</code>).

<code>@Service</code>	Stereotype for a business-logic bean (a specialization of <code>@Component</code>); communicates intent and is the conventional home for transactions.
<code>@Repository</code>	Stereotype for a data-access bean; also translates persistence exceptions into Spring's <code>DataAccessException</code> hierarchy.
<code>@RestController</code>	<code>@Controller</code> + <code>@ResponseBody</code> — return values become the JSON response body.
<code>@Value("\${key}")</code>	Injects a single property value (with optional default after a colon).
<code>@ConfigurationProperties(prefix)</code>	Binds a group of properties to a typed object (used by <code>JwtConfig</code>).
<code>@Autowired</code> / constructor injection	Dependency injection. This project favours <i>constructor injection</i> (final fields, no annotation needed for a single constructor) over field <code>@Autowired</code> — it yields immutable, explicit, test-friendly dependencies.

12.2 Web / MVC

Annotation	What it does & why
<code>@RequestMapping</code>	Class-level base path (and, generally, method+path mappings).
<code>@GetMapping</code> / <code>@PostMapping</code> / <code>@PutMapping</code>	Shorthands binding a method to GET / POST / PUT at a sub-path.
<code>@PathVariable</code>	Binds a URL path segment to a parameter.
<code>@RequestParam</code>	Binds a query-string parameter (supports defaults / optionality).
<code>@RequestBody</code>	Deserializes the JSON request body into a DTO.

<code>@RestControllerAdvice</code>	Global exception-handling advice for all controllers (used by <code>GlobalExceptionHandler</code>).
<code>@ExceptionHandler(Type.class)</code>	Marks a method that handles a specific exception type.

12.3 Security

Annotation	What it does & why
<code>@EnableMethodSecurity</code>	Turns on method-level security so <code>@PreAuthorize</code> is enforced.
<code>@PreAuthorize("hasRole(...)")</code>	Gates a method on a SpEL authorization expression before it runs.

12.4 JPA / Hibernate (Persistence)

Annotation	What it does & why
<code>@Entity</code>	Marks a class as a persistent JPA entity mapped to a table.
<code>@Table(name, indexes)</code>	Specifies the table name and secondary indexes.
<code>@Id</code>	Marks the primary-key field.
<code>@GeneratedValue(generator)</code>	Delegates primary-key generation to a named generator.
<code>@GenericGenerator / @Parameter</code>	Hibernate hook to a custom generator class (here, <code>NumericStringSequenceGenerator</code>) with parameters.
<code>@Column(...)</code>	Maps a field to a column with length / nullable / unique constraints.
<code>@CreationTimestamp / @UpdateTimestamp</code>	Hibernate auto-populates timestamps at insert / on update.

<code>@Transactional</code>	Wraps a method in a database transaction (commit on success, rollback on a runtime exception). Used on multi-write service methods so they are atomic.
-----------------------------	--

12.5 Validation (Jakarta Bean Validation)

Annotation	What it does & why
<code>@Valid</code>	On a controller parameter, triggers validation of the DTO.
<code>@NotBlank</code> / <code>@NotNull</code>	Required value (non-blank string / non-null).
<code>@Email</code>	Must be a valid email address.
<code>@Size(min,max)</code>	Bounds string length.
<code>@Pattern(regex)</code>	Must match a regular expression (phone, role/status whitelists, date format).

12.6 Jackson (JSON)

Annotation	What it does & why
<code>@JsonInclude(NON_NULL)</code>	Omits null fields when serializing (clean envelopes in <code>ApiResponse</code>).
<code>@JsonPropertyOrder</code>	Fixes JSON field order for deterministic output (<code>ErrorResponse</code>).

12.7 Lombok — Note

Lombok is commonly used in Spring projects to remove boilerplate. **This codebase does not use Lombok** — all getters/setters/constructors are hand-written. For completeness: `@Data` generates getters, setters, `toString`, `equals/hashCode`; `@Getter/@Setter` generate accessors; `@Builder` generates a fluent builder; `@NoArgsConstructor/@AllArgsConstructor` generate constructors. None appear in the source.

12.8 Testing Annotations

See [Section 14](#) for the full list (`@Test`, `@BeforeEach`, `@ExtendWith`, `@Mock`, `@InjectMocks`, `@SpringBootTest`, `@WebMvcTest`, `@DataJpaTest`, `@WithMockUser`, etc.).

13. Error Handling, Logging & Configuration

13.1 Error-Handling Strategy

- **Domain exceptions** express business failures (e.g. `DuplicateEmailException`, `AccountSuspendedException`). Business code throws these without knowing about HTTP.
- **One central handler** (`@RestControllerAdvice`) maps each exception to a specific HTTP status and a uniform `ApiResponse.error(message)` body (see [Section 5.2](#)).
- **Validation failures** are handled too: a failed `@Valid` body produces a 400 with all field errors joined into one message.
- **Security failures** are consistent: 401 for missing/invalid tokens (filter + entry point), 403 for failed role checks (`AccessDeniedException`).
- **Anti-enumeration**: login uses the same “Invalid email or password” for both unknown-email and wrong-password, so attackers cannot probe which emails exist.

13.2 Logging Practices

- **Application logging** uses SLF4J (e.g. `DataSeeder` logs seeded departments/admin). SQL logging is controlled per profile (`spring.jpa.show-sql`, `logging.level.org.hibernate.SQL`).
- **Audit logging** is a distinct, business-level concern persisted to the `audit_log` table (not a log file): who did what, in which module, from which IP, when. It is queryable via `GET /iam/auditLogs` for compliance.

13.3 Configuration Files

The app uses Spring profiles so the same build runs differently per environment.

File	Key settings
<code>application.properties</code> (default)	App name; MySQL datasource (<code>...civicdesk?createDatabaseIfNotExist=true</code>); <code>ddl-auto=update</code> ; <code>show-sql=true</code> ; MySQL8 dialect; JWT secret +

	<code>expiry=1800000</code> ms (30 min); admin seed properties (env-overridable); <code>server.port=8081</code> ; <code>context-path=/civicDesk</code> .
<code>application-dev.properties</code>	Verbose development: <code>ddl-auto=update</code> , <code>show-sql=true</code> , <code>format-sql=true</code> , Hibernate SQL at DEBUG.
<code>application-prod.properties</code>	Production-safe: <code>ddl-auto=validate</code> (never auto-create — schema is migration-managed), no SQL logging; datasource and JWT secret come only from environment variables (<code>DB_URL</code> , <code>DB_USERNAME</code> , <code>DB_PASSWORD</code> , <code>JWT_SECRET</code>) — no secrets committed.
<code>application-test.properties</code> (tests)	In-memory H2 with the <code>test</code> profile; same JWT expiry so token math matches.

Every endpoint is served under the context path `/civicDesk` plus the module prefix `/iam`, e.g. `http://localhost:8081/civicDesk/iam/...`. Swagger UI is at `/civicDesk/swagger-ui.html`.

13.4 External Dependencies (Runtime Services)

- **MySQL 8** — the production datastore (auto-created on first run in dev).
- **Environment variables** — in production: `DB_URL/USERNAME/PASSWORD`, `JWT_SECRET`, and the `ADMIN_*` seed values.
- **Docker** — the README documents building/running a container image (`docker run -p 8081:8081 --env-file .env`).

14. Testing (JUnit & Mockito)

14.1 Purpose of Testing & the Test Pyramid

Automated tests verify behaviour, catch regressions, document expected behaviour (test names read like a spec), and let the team refactor with confidence. This module layers its tests into the classic pyramid:

- **Unit tests** — one class in isolation with mocked dependencies (fast, narrow).
- **Slice tests** — load just one layer: `@WebMvcTest` (controllers) or `@DataJpaTest` (repositories against H2).
- **Integration tests** — the full Spring context + real H2 + real security filter chain (broad, end-to-end).

14.2 Testing Annotations Explained

Annotation	Meaning
<code>@Test</code>	Marks a JUnit 5 test method.
<code>@BeforeEach</code> / <code>@AfterEach</code>	Run before / after every test (e.g. seed data; clear the SecurityContext) to keep tests independent.
<code>@ExtendWith(MockitoExtension.class)</code>	Enables Mockito for the test class (initializes mocks, validates stubbing).
<code>@Mock</code>	Creates a fake collaborator whose behaviour you script.
<code>@InjectMocks</code>	Instantiates the class under test and injects the mocks into it.
<code>@SpringBootTest</code>	Loads the full application context (integration test).
<code>@WebMvcTest(X.class)</code>	Loads only the MVC slice for one controller (HTTP, JSON, validation).
<code>@DataJpaTest</code>	Loads only the JPA slice (entities + repositories) against an embedded DB; each test rolls back.
<code>@AutoConfigureMockMvc</code>	Provides <code>MockMvc</code> , a fake HTTP client (with or without the security filters).
<code>@MockitoBean</code>	Places a Mockito mock into the Spring (slice) context, replacing the real bean.
<code>@WithMockUser(roles=...)</code>	Runs a test as a fake authenticated user with given roles.
<code>@ActiveProfiles("test")</code>	Activates the <code>test</code> profile (H2 configuration).
<code>@AutoConfigureTestDatabase(replace=NONE)</code>	Use the configured datasource instead of swapping in a default embedded DB.

14.3 Mockito & Why Mocking Is Required

A mock is a scripted stand-in for a real collaborator. Using real databases/encoders/HTTP objects in a unit test would be slow, non-deterministic, and would blur which class actually failed. Mocking lets you isolate one unit, drive every branch (force a repository to return empty to hit a “not found” path), run fast with no I/O, and verify side-effects. Key patterns used:

- `when(dep.method(args)).thenReturn(value)` — stub a return value.
- `verify(mock).method(args)` / `verify(mock, never())...` / `verifyNoInteractions(mock)` — assert (or refute) that calls happened.
- Argument matchers `any()`, `anyString()`, `eq(value)`.
- `ArgumentCaptor` — capture the object passed into a mock and inspect it (used to verify the `AuditLog` built inside the service).

14.4 The Test Suite, File by File

Test file	Type	What it validates
<code>CivicDeskApplicationTests</code>	Integration smoke	The full Spring context boots (<code>contextLoads()</code>).
<code>NationalIdUtilTest</code>	Pure unit	SHA-256 hashing is deterministic and 64 hex chars; null/blank → null; <code>maskLast4</code> masks all but the last four.
<code>JwtUtilTest</code>	Spring context	Token generation, validation, and role extraction round-trip with the real signing key.
<code>JwtAuthFilterTest</code>	Unit (mocked servlet)	Valid token sets principal/authority and continues; near-expiry token issues a fresh token on the response header; missing header passes through; invalid token → 401 and chain stops.
<code>AuthServiceImplTest</code>	Unit (mocked deps)	Happy-path login returns a token; wrong password, password-not-set, suspended, and wrong-portal cases throw the right exceptions; duplicate-email registration; the full set-password lifecycle.
<code>UserServiceImplTest</code>	Unit (mocked repos)	The creation decision tree: ADM creates DS (and back-links the department); ADM cannot create non-DS; DS requirement of a

		department; DS creating staff inherits its own department; supervisor cannot create admin; duplicate email.
<code>AuditServiceImplTest</code>	Unit (ArgumentCaptor)	<code>log(...)</code> persists an <code>AuditLog</code> carrying exactly the supplied fields.
<code>UserRepositoryTest</code>	Slice (<code>@DataJpaTest</code> , H2)	Derived queries <code>findByEmail</code> / <code>existsByEmail</code> work against a real DB.
<code>AuditLogRepositoryTest</code>	Slice (<code>@DataJpaTest</code> , H2)	Per-user / newest-first queries and specification filters (action, module, case-normalized, combined) return the right rows.
<code>AuthControllerTest</code>	Slice (<code>@WebMvcTest</code>)	Login maps to 200 with the right JSON shape; missing email → 400 (validation) before the service is called.
<code>UserControllerTest</code>	Slice (<code>@WebMvcTest</code> , <code>@WithMockUser</code> <code>ADM</code>)	Admin create → 201; admin list → 200. (403 enforcement is deferred to the RBAC integration test.)
<code>AuditLogControllerTest</code>	Slice (<code>@WebMvcTest</code>)	ADM and CO can read; query params are forwarded to the service (verified with <code>eq(...)</code>).
<code>AuthRequestValidationTest</code>	Slice (validation)	Each invalid field (phone, email, password length, gender, date) yields a precise 400 and never reaches the service; valid/optional-omitted payloads pass.
<code>UserRequestValidationTest</code>	Slice (validation)	Invalid/blank phone, unknown role, malformed email, blank name, invalid status → 400 with the expected message; service untouched.
<code>AuthIntegrationTest</code>	Full stack (real filters + H2)	register → login → access protected endpoint with a real JWT; no token → 401; the full admin-creates-staff → set-password lifecycle.
<code>RbacIntegrationTest</code>	Full stack (real RBAC)	Admin can list users; citizen gets 403; admin creates a supervisor (201) and

		unknown department → 400; no token → 401.
<code>IdGenerationIntegrationTest</code>	Integration	Seeded admin is <code>10000001</code> ; new users get consecutive numeric-string ids; audit ids are numeric; departments use <code>DPT##</code> codes.
<code>AuditLogDataSeeder</code> / <code>CitizenDataSeeder</code> (in <code>test/util</code>)	Operational tooling (not tests)	Standalone JDBC <code>main()</code> utilities that bulk-load 100k audit rows / 50k citizens for local performance testing of the indexed filters.

14.5 Unit vs Integration — A Deliberate Division of Labour

The controller slice tests intentionally *defer* `@PreAuthorize` 403 enforcement to `RbacIntegrationTest`, because `@WebMvcTest` slices do not reliably enforce method security, but the full-stack tests (with the real filter chain) do. This is a good illustration of choosing the right test type for each guarantee.

15. Appendix: Endpoint Reference

All paths are shown after the context path. Full URL = `http://localhost:8081/civicDesk` + path. Every response uses the standard envelope `{ success, statusCode, message, data, timestamp }`.

Method	Path	Access	Notes
POST	<code>/iam/auth/register</code>	Public	Citizen self-registration → 201.
POST	<code>/iam/auth/citizen/login</code>	Public	CIT only (staff → 403); returns JWT.
POST	<code>/iam/auth/staff/login</code>	Public	Staff only (CIT → 403); password-not-set → 403; suspended → 423; returns JWT.
POST	<code>/iam/auth/setPassword</code>	Public	First-time password setup (one-time per account).
POST	<code>/iam/auth/logout</code>	Authenticated	Audit-logged; client clears the token.

GET	/iam/users/me	Authenticated	Current user's own profile.
POST	/iam/users	ADM, DS	ADM → creates DS; DS → creates FO/ENG/CO in own dept.
GET	/iam/users	ADM, DS	ADM: all users; DS: own dept, active only. Filters: role, status, departmentId, page, size.
PUT	/iam/users/{id}/status	ADM	Activate / suspend / deactivate a user.
GET	/iam/auditLogs	ADM, CO	Newest-first. Filters: userId, action, module, page, size.
GET	/iam/departments	ADM, DS	List departments (to pick a valid departmentId).

Roles Reference

CIT Citizen · **FO** Field Officer · **DS** Dept. Supervisor · **ENG** Engineer · **CO** Coordinator/Compliance · **ADM** Admin. These exact codes are the `@PreAuthorize` and `users.role` contract and must not be renamed.

End of document. This reference reflects the source code of the CivicDesk IAM module as analyzed file-by-file. Two minor observations surfaced during analysis: (1) the JWT contains `userId` and `role` claims (plus `iat/exp`) — there is no `email` claim; (2) `ErrorResponse` is defined for cross-module use but the IAM handler currently returns `ApiResponse`.